

PHP programozás

OOP 3. rész - Magic method, Stringable

Rostagni Csaba

2025. december 4.

Tartalom

- 1 Objektum Orientált Programozás
- 2 Composer

Tartalom

- 1 Objektum Orientált Programozás
 - Magic Methods
 - Stringable interface

Magic methods

- A PHP "mágikus metódusai" két darab aláhúzás jellel kezdődnek.

- `__construct()`

- `__destruct()`

- `__call()`

- `__callStatic()`

- `__get()`

- `__set()`

- `__isset()`

- `__unset()`

- `__sleep()`

- `__wakeup()`

- `__serialize()`

- `__unserialize()`

- `__toString()`

- `__invoke()`

- `__set_state()`

- `__clone()`

- `__debugInfo()`

Konstruktor

- A konstruktor PHP-ban az osztály neve helyett a `__construct()` magic method segítségével tudjuk létrehozni
- A régebbi verziókban (PHP3-4) a konstruktor neve megegyezett az osztály nevével
- Amennyiben névteret is használnunk csak a `__construct()` működik (PHP 5.5.3-tól)
- Öröklődés esetén nem hívja meg a szülő konstruktorát automatikusan

Link:

- Konstruktor és Dekonstruktor - php.net

Konstruktor

PHP

```
class Diak1
{
    private $nev;
    private $kor;

    public function __construct($nev, $kor)
    {
        $this->nev = $nev;
        $this->kor = $kor;
    }
}
```

Konstruktor (típusokkal)

PHP

```
class Diak2
{
    private string $nev;
    private int $kor;

    public function __construct(string $nev, int $kor)
    {
        $this->nev = $nev;
        $this->kor = $kor;
    }
}
```

Destruktor

- A destruktor `__destruct()` magic method lesz.
- A destruktor akkor hívodik rá, amikor már nem létezik ráhivatkozás, vagy a szkript futása véget ér (pl. `exit()` függvény hívásakor)
- Destruktorban nem dobunk kivételt!

Link:

- Konstruktor és Dekonstruktor - [php.net](https://www.php.net)

Destruktor példa

PHP

```
class MyDestructableClass
{
    function __construct() {
        print "In constructor\n";
    }

    function __destruct() {
        print "Destroying " . __CLASS__ . " . \n";
    }
}

$obj = new MyDestructableClass();
exit();
```

Eredmény

Destroying MyDestructableClass.

Magic getter és setter

PHP

```
class Auto {  
    private string $gyarto;  
    private string $tipus;  
    private float $fogyasztas;  
  
    public function __construct(string $gyarto,string $tipus,float $fogyasztas) {  
        $this->gyarto = $gyarto;  
        $this->tipus = $tipus;  
        $this->fogyasztas = $fogyasztas;  
    }  
  
    public function __get($nev):mixed {  
        return $this->$nev;  
    }  
  
    public function __set($nev,$ertek):void {  
        $this->$nev = $ertek;  
    }  
}
```

Magic getter és setter használat közben

PHP

```
$audi = new Auto("Audi", "A6",5.8);  
  
echo "{$audi->gyarto} {$audi->tipus} ({$audi->fogyasztas})\n";  
  
$audi->tipus = "A8";  
$audi->fogyasztas = 6.1;  
  
echo "{$audi->gyarto} {$audi->tipus} ({$audi->fogyasztas})\n";
```

Eredmény

```
Audi A6 (5.8)  
Audi A8 (6.1)
```

- A tulajdonságok láthatósága privát
- Kiíráskor és értékadáskor hibát kellett volna dobnia
- Van `__get()` és `__set()` publikus metódusunk

Magic `__get` magyarázat

```
public function __get($nev):mixed {  
    return $this->$nev;  
}
```

PHP

```
$audi->gyarto
```

PHP

- A `$audi->gyarto` meghívásakor a PHP meghívja a `__get()` metódust
- Az elérni kívánt privát tulajdonság a `gyarto`, így `$nev` változó értéke így `"gyarto"` lesz
- A `return $this->$nev` hívásakor a `$nev` helyére behelyettesítődik az értéke, így olyan, mintha a `return $this->gyarto`-t hívtuk volna meg

Magic __set magyarázat

```
public function __set($nev,$ertek):void {  
    $this->$nev = $ertek;  
}
```

PHP

```
$audi->tipus = "A8";
```

PHP

- A `$audi->tipus = "A8"` meghívásakor a PHP meghívja a `__set()` metódust
- `$nev` változó értéke `"tipus"`, míg a `$ertek` változó értéke `"A8"` lesz
- A `$this->$nev = $ertek` hívásakor a `$nev` helyére behelyettesítődik az értéke, így olyan, mintha a `$this->tipus = "A8"`-at hívtuk volna meg

__get() ellenőrzéssel

PHP

```
public function __get(string $nev):mixed
{
    if(property_exists($this, $nev))
    {
        return $this->$nev;
    }
    return null;
}
```

- Amennyiben nem létező változót kérnénk le hibát eredményezne!
- A `property_exists` ellenőrzi a tulajdonság létezését
 - Első paramétere lehet a **vizsgálandó objektum**, osztályon belül saját maga `$this`
 - Második paramétere a **tulajdonság neve** szöveggént
- Amennyiben létezik a tulajdonság adjuk vissza
- Ha nem létezik, akkor legyen a visszaadott érték `null`
- A visszatérési érték típusa `mixed`, ami megengedi a `null` értéket is

__get() ellenőrzéssel

PHP

```
public function __get(string $nev):mixed
{
    if(in_array($nev,["gyarto", "tipus"]))
    {
        return $this->$nev;
    }
    return null;
}
```

- Nem minden esetben kell minden tulajdonságot olvashatóvá tenni
- Csak a tömbben meghatározott tulajdonságok olvashatóak
- Minden más `null` -t ad vissza

MobilePhone osztály

PHP

```
class MobilePhone {  
    private $manufacturer;  
    private $model;  
    private $year;  
  
    public function __construct(string $manufacturer, string $model, int  
        ↪ $year) {  
        $this->manufacturer = $manufacturer;  
        $this->model = $model;  
        $this->year = $year;  
    }  
  
    public function __get(string $property): mixed {...}  
}
```


MobilePhone __get()

PHP

```
public function __get(string $property): mixed {  
    if ($property === 'age') {  
        return date('Y') - $this->releaseYear; // Calculate age dynamically  
    }  
    if (in_array($property, ['manufacturer', 'model'])) {  
        return $this->$property;  
    }  
    if (!property_exists($this, $property)) {  
        throw new Exception("Ismeretlen tulajdonság: '$property'.");  
    }  
    throw new Exception("A(z) '$property' tulajdonság nem olvasható.");  
}
```

- Mivel az age tulajdonság nem létezik, így a `property_exists` előtt kell visszaadni

MobilePhone használata

PHP

```
$phone = new MobilePhone('Apple', 'iPhone 12', 2020);  
echo $phone->manufacturer;    // Apple  
echo $phone->model;            // iPhone 12  
echo $phone->age;               // 5 (2025-ben)  
echo $phone->year;
```

- `year` tulajdonság lekérésekor kivételt dob:
"A(z) '`year`' tulajdonság nem olvasható."

Tartalom

- 1 Objektum Orientált Programozás
 - Magic Methods
 - Stringable interface

Stringable

PHP

```
interface Stringable {  
    /* Methods */  
    public __toString(): string  
}
```

- Így jelöljük, hogy az osztálynak tartalmaznia kell a `__toString()` metódusal
- A legtöbb interface-től eltérően implicit használja, ha a metódus meglet valósítva
- Elvárás, hogy legyen explicit megadva

Linkek:

- [Stringable.](#) - PHP dokumentáció

A Stringable interface használata

PHP

```
class Car implements Stringable {  
    public function __toString(): string {  
        return "Ez egy autó objektum.";  
    }  
}
```

- Az `implements` után kell megadni az interface-eket
- Több interface is megadható vesszővel felsorolva
- Az osztály innentől kötelezően tartlamazzon egy `__toString` metódust
- A visszatérési típusa `string` legyen

Tartalom

- 1 Objektum Orientált Programozás
- 2 Composer

Tartalom

2 Composer

- composer.json - autoload
 - composer dump-autoload

composer.json - autoload

composer.json

```
{  
    "autoload": {  
        "psr-4": {  
            "Acme\\": "src/Acme"  
        }  
    }  
}
```

- A különböző autoload szabályokat lehet felvenni
- Jelenleg a projektekhez a "psr-4" alkalmazandó
- Meghatározható, hogy melyik névtér kódját hol keresse
 - A kulcs a névtér megnevezése
 - A névtéreket \ jellel kell elválasztani, itt ezt a \\ jelenti
 - A névtér végét mindig le kell zárni backslash karakterrel
 - Az érték a mappa szerkezet a projektkönyvtártól meghatározva, a példában src/Acme. (itt slash karakterrel kell lezárni)

Linkek:

- Basic usage: Autoloading - getcomposer.org

composer dump-autoload

```
composer dump-autoload
```

CMD

- Újra generálja az autoloadhoz szükséges fájlokat
- A `composer.json` módosítása után célszerű futtatni

```
composer dump-autoload -o
```

CMD

- Optimalizálja a fájlokat
- Gyorsítja az oldal működését
- Jelzi, ha a saját fájljaink nem felelnek meg a szabványnak